DUOC UC - ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

Especificación de Requisitos de Software

Proyecto:

Holding Cars - Sistema Digital de Actas

Revisión*:* 2.0
25/10/2026

Especificación de Requisitos según estándar de IEEE 830.

-

Contenido

Ficha del documento **3**
1. Introducción **4**
1.1. Propósito 4
1.2. Ámbito del Sistema 4
1.3. Definiciones, Acrónimos y Abreviaturas 4
1.4. Referencias 4
1.5. Visión General del Documento 4
2. Descripción General **5**
2.1. Perspectiva del Producto 5
2.2. Funciones del Producto 5
2.3. Características de los Usuarios 5
2.4. Restricciones 5
2.5. Suposiciones y Dependencias 5
2.6. Requisitos Futuros 5
3. Requisitos Específicos **6**
3.1 Requisitos comunes de las interfaces 6
3.1.1 Interfaces de usuario 6
3.1.2 Interfaces de hardware 6
3.1.3 Interfaces de software 6
3.1.4 Interfaces de comunicación 6
3.2 Requisitos funcionales 6
3.3 Requisitos no funcionales 7
3.3.1 Requisitos de rendimiento 7
3.3.2 Seguridad 7
3.3.3 Fiabilidad 7
3.3.4 Disponibilidad 7
3.3.5 Mantenibilidad 7
3.3.6 Portabilidad 7
3.4 Otros Requisitos 7
3.5 Actualización de Requisitos - Integración de Framework Frontend (React) 8

-

Ficha del documento

Fecha	Revisión	Autor	Modificación
25/10/2026	2.0	Andrea Mascareño	propuesta personalizada

Documento validado por las partes en fecha:

Por el cliente

[Firma]

Sr./Sra.

Por la empresa suministradora

[Firma]

Sr./Sra.

-

1. Introducción

1.1. Propósito

El propósito de este documento es definir de manera clara y detallada los requisitos funcionales y no funcionales del sistema "Holding Cars". El documento sirve como guía para validar que la interfaz web (HTML, CSS, JS) cumple con las necesidades de digitalización de actas, operando con datos locales (LocalStorage) y preparada para una futura conexión con el Backend de microservicios.

1.2. Ámbito del Sistema

- **Nombre del sistema:** Holding Cars - Sistema de Gestión de Actas.
- **Lo que el sistema hará (Frontend):** Permitirá a los usuarios iniciar sesión (simulado), visualizar la flota de vehículos disponible en tarjetas o tablas, registrar el estado de entrega/recepción de un vehículo (kilometraje, combustible, daños) y almacenar estos datos en el navegador (LocalStorage). Incluirá mantenedores visuales de usuarios y vehículos.
- **Lo que el sistema NO hará (en esta entrega):** No se conectará a una base de datos MySQL real todavía (se simula con JS y LocalStorage)
- **Beneficios:** Interfaz de usuario moderna, responsive (adaptable a móviles), validación de datos en el lado del cliente para evitar errores y navegación fluida entre las distintas vistas.

1.3. Definiciones, Acrónimos y Abreviaturas

- **ERS:** Especificación de Requisitos de Software.
- **Frontend:** Capa visual del sistema (HTML, CSS, JavaScript).
- **LocalStorage:** Almacenamiento local del navegador web utilizado para simular la persistencia de la base de datos.
- **Responsive:** Capacidad del diseño para adaptarse a pantallas de celulares y computadoras.
- **XAMPP:** Entorno local utilizado para pruebas de servidor (Apache/MySQL).

1.4. Referencias

- README del repositorio Frontend en GitHub.
- Planilla de Requerimientos (Excel).
- Repositorio GitHub: https://github.com/JohaSo/Proyecto_Holing_Cars_Back-end.git

1.5. Visión General del Documento

Este documento detalla el contexto del sistema, las características de los usuarios, los requisitos de interfaz y el listado exhaustivo de los 15 requerimientos (funcionales y no funcionales) implementados en el Frontend.

-

2. Descripción General

2.1. Perspectiva del Producto

Este producto es la interfaz de usuario (Frontend) del sistema Holding Cars. Actualmente funciona de manera independiente y local, utilizando arreglos de JavaScript y localStorage para simular el comportamiento del servidor. En un futuro, consumirá los endpoints del API Gateway del backend de microservicios (Spring Boot).

2.2. Funciones del Producto

- **Módulo de autenticación:** Login con validación de correo.
- **Módulo de Vehículos:** Visualización de vehículos disponibles para el movimiento solicitado.
- **Módulo de Navegación:** Menú superior para moverse entre Inicio, Vehículos, Nosotros y Login.

2.3. Características de los Usuarios

- Administrador: Encargado de la gestión de la empresa. Nivel técnico medio. Necesita ver reportes, gestionar usuarios y agregar vehículos a la flota.
- Chofer: Usuario final que retira o entrega el auto. Nivel técnico básico. Necesita una interfaz muy intuitiva (idealmente en móvil) para llenar el acta sin errores.

2.4. Restricciones

- Tecnológicas: El desarrollo se limita a HTML, CSS y JavaScript, sin frameworks de Frontend (React, Angular, Vue).
- Hardware: El diseño debe ser adaptable a dispositivos móviles (Tablets y celulares) para que el chofer pueda registrar el acta in situ.
- Red: Para pruebas locales del proyecto completo, se requiere XAMPP.

2.5. Suposiciones y Dependencias

- Se asume que el navegador del usuario soporta localStorage y JS.
- La interfaz depende de que los datos simulados en JS estén correctamente ingresados.

2.6. Requisitos Futuros

- Conexión real con el API Gateway y los 15 microservicios.
- Llenado de acta desde la web a realizar.
- Generación de documentos PDF de las actas.
- Envío de notificaciones por correo electrónico.

-

3. Requisitos Específicos

3.1 Requisitos comunes de las interfaces

3.1.1 Interfaces de usuario

Las interfaces serán páginas web responsive con un menú de navegación superior y un área de contenido central. En la zona de administración se utilizará un menú lateral. Los colores corporativos serán azul oscuro, gris claro y blanco.

3.1.2 Interfaces de hardware

El sistema debe ser 100% responsivo. Se debe poder utilizar en pantallas de escritorio (resolución 1920x1080) y en dispositivos touch (Tablets de 10 pulgadas o smartphones) para el llenado del acta.

3.1.3 Interfaces de software

- Navegador Web: Brave.
- Almacenamiento: API de localStorage del navegador para persistir actas y usuarios.

3.1.4 Interfaces de comunicación

Por el momento, no hay comunicación con servidores externos (API). Toda la comunicación es interna del navegador (DOM Events). Cuando se integre el backend, la comunicación será vía HTTP con formato JSON.

3.2 Requisitos funcionales

- **3.2.1 Requerimiento funcional 1: Acceso al Sistema (Login):**
  - **Actores:** Administrador, Chofer
  - **Descripción:** El sistema permite el acceso mediante correo y contraseña.
  - **Validaciones:** El correo debe cumplir el regex de dominios permitidos (@duoc.cl, @profesor.duoc.cl, @gmail.com). La contraseña debe tener entre 4 y 10 caracteres. Al ser exitoso, guarda sesión en localStorage y redirige al panel.
- **3.2.4 Requerimiento funcional 4: Visualización de Flota de Vehículos**
  - **Actores:** Chofer, Administrador.
  - **Descripción:** El sistema muestra un listado en formato de tarjetas o tabla con los vehículos.
  - Implementación: Las tarjetas se generan únicamente si antes el Chofer/Administrador inicia sesión de manera exitosa. Cada tarjeta muestra Imagen,Patente,Marca,Modelo,Kilometraje y Estado (Disponible/En Retiro). Si no está disponible, el botón está deshabilitado.
- **3.2.11 Requerimiento funcional 11: Navegación entre Páginas**
  - Actores: Administrador, Chofer
  - Descripción: Hipervínculos coherentes entre vistas públicas.
  - Implementación: Los enlaces del menú redirigen a Inicio, Vehículos, Nosotros, etc.

-

3.3 Requisitos no funcionales

3.3.1 Requisitos de rendimiento

- La interfaz debe cargar los datos de la flota y usuarios de manera rápida al utilizar arreglos locales en JavaScript.
- La lista de vehículos debe cargarse en menos de 1 segundo al abrir la página.
- No debe haber errores en la consola del navegador al ejecutar el JS.

3.3.2 Seguridad

- Control de Acceso: El sistema restringe el acceso a las vistas de administración (ej. vehículos.html) solo para usuarios autorizados. Si no hay sesión activa en el localStorage, el código redirige automáticamente al login.html.
- Validación de Entradas: validación por medio de Auth en JS para validar longitud de contraseñas antes de procesar los datos.

3.3.3 Fiabilidad

El sistema debe manejar de forma controlada los errores de validación (ej. Correo incorrecto) mostrando mensajes claros al usuario sin que la página se caiga o se recargue.

3.3.4 Disponibilidad

El frontend está disponible en un 100% mientras el navegador web esté abierto y el usuario tenga conexión local a los archivos del proyecto.

3.3.5 Mantenibilidad

- Estructura: El código está estructurado en carpetas separadas (css/, js/, html/, img/) y cuenta con comentarios explicativos para facilitar futuras modificaciones.
- Estilos: Uso de variables CSS para facilitar el cambio de la paleta de colores si la empresa lo requiere.

3.3.6 Portabilidad

Al estar desarrollado con tecnologías web estándar (HTML, CSS, JS), el sistema es portable a cualquier sistema operativo (Windows, Mac, Linux) y navegador moderno, sin necesidad de instalación.

3.4 Otros Requisitos

Diseño Responsive y Consistente: El sistema debe presentar un diseño visual coherente, adaptable a diferentes tamaños de pantalla. El menú y las tarjetas deben reorganizarse correctamente en pantallas móviles

-

3.5 Actualización de Requisitos - Integración de Framework Frontend (React)

- Requerimiento Funcional (Navegación): Se implementó React Router DOM para permitir la navegación entre las distintas vistas de la aplicación (Home, Ofertas, Nosotros, Contacto, Login y Registro) sin recargar la página.
- Requerimiento Funcional (Componentes): La vista de catálogo de vehículos se dividió en componentes reutilizables (VehiculoCard), los cuales reciben propiedades (props) para mostrar la información de cada auto.
- Requerimiento Funcional (Estados): Se utilizó el hook useState en el componente Home para gestionar el estado del vehículo seleccionado por el usuario, mostrando una alerta dinámica.
- Requerimiento No Funcional (Diseño Responsivo): Se integró el framework CSS Bootstrap (mediante react-bootstrap) para garantizar que las tarjetas de vehículos, los formularios y el menú de navegación se adapten correctamente a pantallas de celulares, tablets y computadoras (uso de Container, Row y Col).
- Requerimiento No Funcional (Testing): Se configuró un entorno de pruebas unitarias utilizando Jasmine y Karma para validar el correcto renderizado del componente VehiculoCard.